

A Mini-Project -1 Report on
FINBUDDY AI
AI Powered Personal Finance Tracker

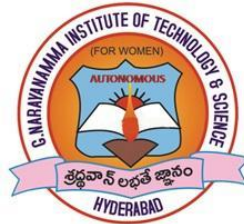
**Submitted to the Department of Computer Science & Engineering, GNITS in the
partial fulfillment of the academic requirement for the award of B.Tech (CSE)
under JNTUH**

By

Aatifa Tahmeed (241251A05A0)
Samreen Kausar (241251A05C6)
Sk. Musqan Khadri (241251A05C7)

under the guidance of

Dr.Jayashree S Patil
Associate Professor



Department of Computer Science & Engineering
G. Narayanamma Institute of Technology & Science

Autonomous (For Women)

Shaikpet, Hyderabad 500 104.

Affiliated to
Jawaharlal Nehru Technological University Hyderabad

Hyderabad – 500 085

June 2026

G. Narayanamma Institute of Technology & Science

AUTONOMOUS

(For Women)

Shaikpet, Hyderabad – 500 104.

Department of Computer Science & Engineering



Certificate

This is to certify that the Mini-Application 1 report on “FinBuddy AI – AI-Powered Personal Finance Tracker” is a Bonafide work carried out by Aatifa Tahmeed (24251A05A0), Samreen Kausar (24251A05C6), Sk. Musqan Khadri (24251A05C7) in the partial fulfillment for the award of B.Tech degree in Computer Science & Engineering, G. Narayanamma Institute of Technology & Science, Shaikpet, Hyderabad, affiliated to JNTUH, Hyderabad under our guidance and supervision.

The results embodied in the Application work have not been submitted to any other University or Institute for the award of any degree or diploma.

Internal Guide

Dr. Jayashree S Patil

Associate Professor

Department of CSE

Head of the Department

Dr. A. Sharada

Professor & Head

Department of CSE

G. Narayanamma Institute of Technology & Science
AUTONOMOUS (For Women)
Shaikpet, Hyderabad – 500 104.

Department of Computer Science & Engineering

Research Center for Cloud Computing

Certificate

This is to certify that Aatifa Tahmeed (24251A05A0), Samreen Kausar (24251A05C6), Sk. Musqan Khadri (24251A05C7) of II B.Tech, has successfully completed Application work in **“Research Center for Cloud Computing”** CSE Department, as part of ongoing research activities carried out in research centre in collaboration with **“AION-TECH Solutions Limited”**, Hyderabad.

The application titled **“FinBuddy AI – AI Powered Personal Finance Assistant”** that is being submitted in partial fulfillment for the award of B.Tech, Computer Science and Engineering , G. Narayanamma Institute of Technology & Science affiliated to JNTUH, Hyderabad is a record of Bonafide work carried out by them in our guidance and supervision.

Internal Guide

Dr. Jayashree S Patil

Associate Professor

Department of CSE

RC In charge

Mrs. Bhageshwari Ratkal,

Assistant Professor,

Department of CSE

Head of the Department

Dr. A. Sharada,

Professor & Head

Department of CSE

Acknowledgements

We would like to express our sincere thanks to **Dr. K. Ramesh Reddy**, Principal, GNITS, for providing the working facilities in the college.

Our sincere thanks and gratitude to **Dr. M. Seetha**, Professor CSE, Dean R&D, **Dr. N. Kalyani**, Professor CSE, Dean I&I, GNITS, for all the timely support and valuable suggestions during the period of our application.

We extend our heartfelt gratitude to **Dr. A. Sharada**, Professor & Head, Department of Computer Science and Engineering, GNITS, for her unwavering support and invaluable guidance throughout our application, providing timely assistance and insightful suggestions.

We are extremely thankful to **Dr. P. Sunitha Devi**, Assistant Professor Mini-Project 1 overall coordinator, Department of CSE, GNITS for all the valuable suggestions and guidance during the period of our application.

We are also extremely thankful to our application coordinators **Mrs. Bageshwari Ratka**, Assistant Professor, **Mrs. K.Sindhura**, Assistant Professor and **Mrs. G. Amulya**, Assistant Professor Dept. of CSE, GNITS, for their encouragement and support throughout the Application.

We are extremely thankful and indebted to our internal guide, **Dr. Jayashree S Patil**, Associate Professor, Department of CSE, GNITS, for her constant guidance, encouragement, and moral support throughout the Application.

Finally, we would also like to thank all the faculty and staff of CSE Department who helped us directly or indirectly, parents and friends for their cooperation in completing the Application work.

Aatifa Tahmeed (24251A05A0)

Samreen Kausar (24251A05C6)

Sk. Musqan Khadri (24251A05C7)

ABSTRACT

Financial literacy and personal budget management are challenges faced by millions of people all around the world. Without a clear view into spending habits, savings potential, or budgeting, people are unable to make informed financial decisions. FinBuddy AI is an intelligent, web-based finance tracker designed to address these challenges by combining full-stack development with Artificial Intelligence (AI). This system allows its users to log, track, and analyze their financial decisions in real time. Users interact with the built-in AI chat assistant powered by the Groq Application Programming Interface (API) using the LLaMA 3.3 70B Large Language Model (LLM), to ask questions about their spending, receive personalized budget advice, and log expenses via voice or text input in natural language. The application also provides automated budget alerts at predefined thresholds, specifically at 80% and 100% of the monthly budget, delivering in-app notifications and email warnings via the Resend email service.

The frontend is developed using React, TypeScript, and Vite, with Tailwind Cascading Style Sheets (CSS) for responsive styling. Supabase provides the backend, which provides a managed PostgreSQL database, authentication, realtime subscriptions, and serverless edge functions built on the Deno runtime. The system architecture is completely cloud-based and deployed on Vercel, ensuring high availability and scalability.

Keywords: React, TypeScript, Supabase, PostgreSQL, Groq API, LLaMA, AI Chat Assistant, Personal Finance, Budget Tracker, Realtime Notifications, Edge Functions, Tailwind CSS.

Sustainable Development Goals (SDGs):

SDG 1 – No Poverty: by empowering individuals with financial awareness tools to manage income and reduce unnecessary expenditure;

SDG 8 – Decent Work and Economic Growth: by enabling better personal financial planning and savings management;

SDG 10 – Reduced Inequalities: by providing accessible, AI-powered financial guidance to users regardless of their background.

Table of Contents

Sl. No.	Topic	Page No.
	Abstract	vi
	List of Figures	ix
	List of Tables	x
1.	Introduction	1
	1.1 Background of the Study	1
	1.2 Problem Statement	1
	1.3 Existing Systems	1
	1.4 Drawbacks of Existing System	2
	1.5 Proposed System	2
	1.6 Objectives	2
	1.7 Methodology	3
	1.8 Hardware and Software Requirements	3
	1.9 Organisation of the Application	4
2.	Literature Survey	5
3	Architecture for FinBuddy	8
	3.1 System Architecture	8
	3.2 Module Design for FinBuddy	9
4.	Implementation of FinBuddy	10
	4.1 Tools and Technologies Used	10
	4.2 Schema Design of Database	11
	4.3 API Architecture and Implementation	12
	4.4 Module-wise Implementation	13
	4.5 Report Generation Workflow	15
	4.6 Security and Data Integrity	15
5.	Results and Discussions	17
	5.1 Results of Each Module	17
	5.2 Performance Measures	19
	5.3 Testing and Validation	20
6.	Conclusion and Future Enhancements	22
	6.1 Conclusion	22
	6.2 Future Scope	22

References	24
Glossary	26
Appendix	29

List of Figures

Figure No.	Figure Name	Page No.
1.1	Methodology	3
3.1	System Architecture	8
3.2	Module Design	9
4.1	React Component Hierarchy	10
4.2	Database Entity Relationship Diagram	12
4.3	AI Chat Expense Logging Flow	14
4.4	Budget Alert Trigger Pipeline	14
5.1	Dashboard	17
5.2	Forecasts Page	18
5.3	AI Chat Assistant Interaction	18
5.4	Overspending Alert Sample	19
5.5	Predictions Page	19

List of Tables

Table No.	Table Name	Page No.
2.1	Literature Survey Comparison Table	7
4.1	Technologies	10
4.2	Database Tables – Columns and Data Types	12
5.1	Performance Analysis of System Modules	20
5.2	Unit Testing Results	20
5.3	Integration Testing Results	21

1. INTRODUCTION

1.1 Background of the Study

Financial management is a very important life skill, yet many people lack structured tools to track and control their daily spendings. In the digital age it becomes very difficult to manually track where the money is going. This information gap leads to overspending, poor financial management.

Traditional financial management tools such as spreadsheets and google workbooks rely heavily on manual entry. Existing applications are either too basic or commercial and complex for everyday use. These systems focus on recording only past transactions and provide limited analytical or predictive capabilities. As a result, users struggle to effectively monitor spending and control budgets.

FinBuddy AI addresses this gap by providing a web-based platform that combines real-time transaction with an AI- powered chat assistant. It replaces manual entry with NLP (Natural language parser) and converts static transaction history into predictive intelligence. It shifts the user experience from just a digital tracker to Intelligent assistance. The application is built on modern, cloud-native technologies and is designed in such a way that it can be easily used by the users of all technical backgrounds.

1.2 Problem Statement

Individuals usually struggle to maintain their budget or track their spending across multiple categories and payment methods. Without a personalized system over-spending is usually detected by end of the month by then it is too late to take an action. Existing finance management tools such as Excel-sheet or basic apps lack intelligent insights, automated categorization, or early budget warnings. As a result, users struggle to effectively monitor spending, control budgets, and make informed financial decisions.

1.3 Existing System

Several Financial management apps exist in today's market. They primarily consist of excel sheets, google notebooks, basic tracking apps and certain apps such as Mint (by Intuit) provides bank synchronization and spending categorization but is limited to the United States and does not offer conversational AI. YNAB (You Need a Budget) focuses

on manual budgeting methodology but lacks automated transaction import and AI assistance. None of these systems combine real-time transaction logging, an AI chat assistant with full transaction context, voice input, and automated email alerts in a single accessible platform.

1.4 Drawbacks of Existing System

- Requires continuous manual entry
- No intelligent or automated categorization
- No real time AI insights that understand user behavior
- High possibility of human error
- Does not detect abnormal spending
- Lack of multicurrency in most applications
- Limited personalized financial insights

1.5 Proposed System

FinBuddy AI is a full stack application utilizing AI for personal finance tracking purposes. The app allows a user to log transactions using a chat interface either in text or through voice communication, analyze spending trends in real-time, provides personalized budget recommendations generated by an AI based on user's past transactional data and automatically triggers email notifications once 80% or 100% of user's budget has been exhausted for that month. Figure 1.1 shows the Systems Architecture

The app is deployed on Vercel and runs on Supabase, a fully managed PostgreSQL database, authentication and serverless edge functions. The AI chat assistant passes the user's message along with all their transaction history to the Groq API (LLaMA 3.3 70B model), receiving context-aware responses back. The app supports different currency options, dark and light mode, voice logging for expenses and interactive visualizations such as pie charts and stacked bar graphs.

1.6 Objectives

- To design an intelligent language input system that enables users to record expenses using natural language and automatically extract transaction details.
- To develop an actionable financial feedback mechanism that provides meaningful insights and recommendations based on user spending patterns and implement behavioral alerts that notify users when abnormal or excessive spending is detected.

- To analyze daily spending trends in order to perform predictive expense estimation for forecasting end-of-month expenditures.
- To develop a personalized budget coaching module that guides users in improving financial discipline and achieving savings goals.
- Deploy the system as a publicly accessible cloud application on Vercel.

1.7 Methodology

The development methodology was based on an iterative process with a phased approach. Phase 1 involved requirements gathering and technology selection, which yielded the decision to use React and TypeScript for the frontend, Supabase for the backend, and Groq for AI functionality. Phase 2 dealt with database design, which comprised the definition of three critical tables – transactions, profiles, and notifications along with proper Row Level Security (RLS).

Phase 3 consists of the actual development of the core application, which included the authentication process, transaction CRUD operations, the implementation of React Query, the metric cards on the dashboard, and the navigation framework. Phase 4 involved integrating the AI chat assistant, which entailed designing a system prompt incorporating transaction context in all AI calls and a custom structured response block protocol for expenses, charts, and tables.

Phase 5 entailed the design and implementation of the budget alert system, involving the PostgreSQL database trigger on all transaction inserts, the Hypertext Transfer Protocol (HTTP) call from the trigger using `pg_net`, and the Resend email integration. Finally, Phase 6 covered frontend refinements such as the stacked bar chart, the teal-orange color scheme, voice input, and dark mode.

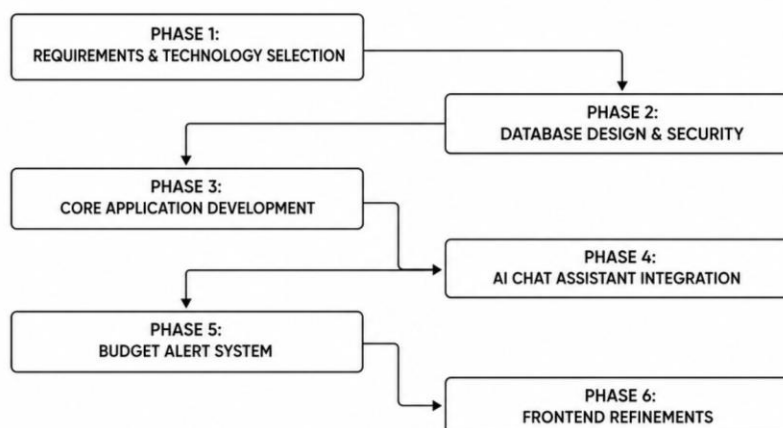


Figure 1.1 Methodology

1.8 Hardware and Software Requirements

Hardware:

- A desktop PC or laptop with at least 4 Gigabytes (GB) RAM.
- Fast internet connectivity (minimum of 10 Mbps is advised).
- Microphone (optional, to use the voice input functionality).

Software:

- React 18.3 with TypeScript – Frontend framework.
- Vite 5.4 – Build tool and development server.
- Tailwind CSS – Utility First CSS framework.
- Supabase – Backend with features such as Postgres database, authentication, real-time support, and edge functions.
- Groq API (LLaMA 3.3 70B) – AI-powered language model for the chatbot.
- Resend API – Email delivery service for budget reminders.
- TanStack Query (React Query) – Server-side state management and data caching.
- Recharts – Charting framework for displaying analytics.
- Vercel – Deployment service.
- Node.js v18 or above and npm.

1.9 Organization of the Application

The organizational structure of the Application report is presented below. The second chapter consists of the review of relevant literature, which includes an analysis of current literature on personal finance management, AI chatbots, and financial websites. Chapter 1 provides the introduction. The Literature Survey is given in Chapter 2 .The system design is explained in Chapter 3, including its architecture and modularity. Chapter 4 gives the details of system implementation, including technology and tool usage, database schema, API creation, modular implementation, report generation process, and security. Chapter 5 provides the results of the experiment along with system performance and comparison . Chapter 6 provides the conclusion.

2. LITERATURE SURVEY

Existing literature was thoroughly reviewed to understand the current state of applications for personal financial management, AI-powered financial advisors, etc., and the technologies used for them. This survey identifies the gaps that FinBuddy AI is designed to fill.

Early personal finance tools like Mint and Quicken rely on rule-based categorization engines. Transactions taken from bank data were matched with keyword databases to assign spending categories. These systems were useful for users with supported bank accounts, but included no conversational intelligence other than categorization. Users' feedback generally and consistently showed a demand for interactive guidance and proactive advice instead of just a set of passive dashboards [1].

Later research explored the usage of Machine Learning (ML) classifiers to categorize transactions. Naive Bayes and Support Vector (SVM) classifiers were applied to merchant name strings to categorize spending, even achieving accuracy rates above 85% on labelled datasets [2]. These models, however, required labelled training data as well as periodic retraining with the evolution of merchant patterns. FinBuddy AI instead uses a hybrid approach, combining a Large Language Model (LLM) for contextual understanding and a keyword-based client-side parser for instant categorization.

With the advent of Large Language Models (LLMs), there are new opportunities in providing personalized financial advice. According to Kearns and Nevmyvaka (2013), an example of the effectiveness of algorithms used in finance was established [3]. More recently, experiments using the large language models, GPT-4 and LLaMA, for financial Q&A tasks prove the ability of the models to engage in reasoning about structured financial data in a human-friendly way [4]. The FinBuddy AI approach takes this one step further and embeds all user transactional, category information, and anomaly information in the prompt itself.

Recent research has demonstrated that people are considerably more inclined to use financial apps if they include natural language processing features [5]. In addition, voice-operated CUIs facilitate access to such applications for people with poor typing skills or mobile users. A study has discovered that people receiving real-time notifications on their

expenditure were 34% less likely to exceed their monthly budget than the control groups [6]. The use of the Web Speech API in FinBuddy AI's chat interface is directly inspired by these results.

The development of Backend-as-a-Service (BaaS) platforms has made building scalable web apps considerably easier. Supabase, which is built upon the combination of PostgreSQL and the PostgREST API layer, allows for the use of auto-generated REST APIs, JWT-based authentication, row-level security, and live updates via logical replication [7]. This means that there is no need for a custom API server while ensuring the security of an enterprise-level system. In FinBuddy AI, these features are used in addition to serverless edge functions for AI and emails.

Edge functions powered by Deno represent an efficient, high-performance, low-cost alternative to serverless cloud solutions [8]. Using Supabase Edge Functions that enable FinBuddy AI's AI chat, spend prediction engine, and email notification pipeline to be run within milliseconds without scaling issues. Thus, it is possible to avoid building a custom server infrastructure, saving considerable amounts of money on deployment.

A recent study found that interactive graphs considerably improve understanding of financial situations [9]. In particular, stacking graphs proved helpful in analyzing the categorization of expenditure per day. Accordingly, FinBuddy AI uses stacked bar graphs on its Forecasts page through the library Recharts, with colors matching the pie chart legend to ensure consistency.

Row Level Security (RLS) is a security mechanism of the PostgreSQL database engine that operates independently from application-level filters. In research on multi-tenant SaaS software applications, researchers found that RLS offers a more effective protection against data leakages than application-level filtering. FinBuddy AI employs RLS for all three database tables by applying policies that compare the JWT token of the currently authenticated user against the value in the row's `user_id` column.

With event-driven software architectures, business logic operations may be triggered by data changes without changing any application logic. The usage of `pg_net` to make HTTP requests in PostgreSQL triggers allows FinBuddy AI's budget alert system to

operate inside the database layer. This design guarantees that notifications will be delivered to users regardless of whether they use FinBuddy AI at the moment [12].

From the analysis of the literature, one can conclude that although all four components of FinBuddy AI are present separately in various products, there is currently no publicly available software that incorporates all four together: AI-powered chat with complete transaction information, budget alerts, analytics dashboard with stacked visuals, and voice interaction. Table 2.1 shows the literature Survey Comparison Table

Table 2.1 Literature Survey Comparison Table

S.No	Title of the Journal with Published Year	Author's	Method	Outcomes	Limitations
1	AI-Powered Personal Finance Management (2026)	Global FinTech Research	Agentic AI & Generative Workflows	Shift to outcome-driven automation	High computer needs.
2	Smart AI-Based Personal Finance Assistant (2025)	Sharma, R. et al.	Hybrid Model (BERT + LSTM)	High-accuracy expense categorization	performance relies heavily on the quality of labeled data.
3	Smart Spend: ML for Predictive Budgeting (2025)	Chen, L. & Wu, H.	NLP-based receipt parsing & K-Means Clustering	95% accuracy in automated categorization and real-time alerts.	Difficulty parsing handwritten or low-quality physical receipt images.
4	Personal Finance Management using Flask (2025)	Verma, A.	Linear Regression & Recommendation Engines	Automated tracking with investment and SIP suggestions.	Struggles with volatile data.

3. ARCHITECTURE FOR FINBUDDY

3.1 System Architecture

The FinBuddy AI system follows a four-layer cloud-native architecture as illustrated by Figure 3.1 below. The Presentation layer includes the React & TypeScript frontend served over Vercel CDN. The Application Layer involves the use of Supabase Edge Functions, which run on the Deno runtime environment and perform API calls, emails and spend predictions. The Data layer contains the managed PostgreSQL database hosted on Supabase, which comprises three main tables and RLS policies for each. The External Services Layer involves integration with the Groq API for LLM inference and the Resend API for sending transactional emails.

The frontend communicates with the backend over HTTPS. Supabase JavaScript client manages authentication via the use of JWT tokens, database querying via the PostgREST API and WebSocket for real-time subscriptions. Edge Functions are invoked through fetch calls where the JWT token is passed as a bearer token.

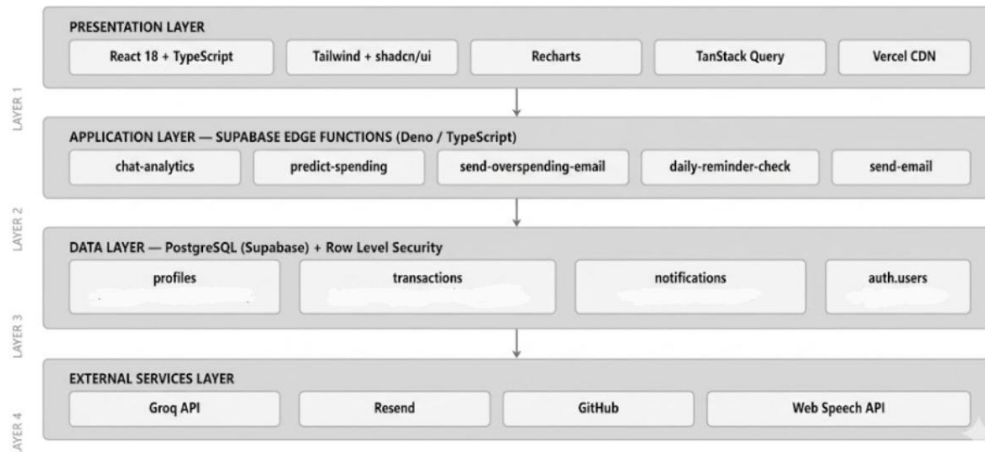


Figure 3.1 System Architecture

3.2 Module Design Of FinBuddy

The application consists of six main modules as indicated by Figure 3.2 below. The Authentication module is responsible for registering users and logging them in via the Supabase Auth service. This module stores JWT tokens in local storage and also auto-refreshes them to prevent expiry. The Transactions Management Module provides CRUD

operations for transactions via React Query hooks with automatic cache invalidation on every mutation.

The AI Chat module sends messages along with the last 10 messages of the conversation history to the chat analytics edge function. In return, the module gets structured responses which include any expense, chart or table blocks, followed by the confirmation process for the expense entry. The Budget alert Module is made up of a PostgreSQL Trigger, a pg_net HTTP call and two edge functions (send overspending email and send email). The module provides in-app and email notifications.

The Analytics Module computes all charts and statistics through client-side memorized computations of the transaction data. The notifications module involves a WebSocket real-time subscription to the database trigger that adds new rows to the notifications table.

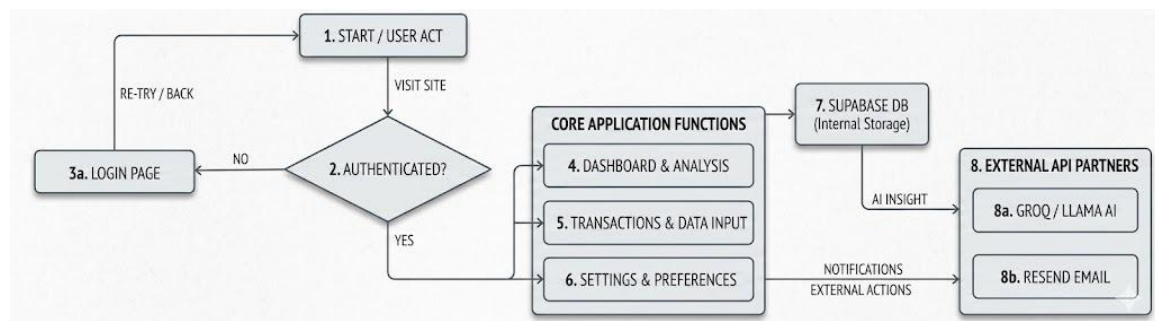


Figure 3.2 Module Design

4. IMPLEMENTATION OF FINBUDDY

4.1 Tools and Technologies Used

The development of FinBuddy AI required the use of a particular set of tools and technologies. Frontend is built with React 18.3 framework and TypeScript, while Vite 5.4 handles bundling. Navigation in the React application is implemented with React Router 6 and uses wrappers for protected routes. The state of the server is managed with the help of TanStack Query (React Query), with user-specific caching keys used. There are three global context providers, namely AuthContext, ThemeContext and CurrencyContext, nested in the root of the application to avoid prop-drilling. All styles were added using Tailwind CSS only.

Supabase serves as the entire backend, providing a managed PostgreSQL database, JWT authentication, row-level security, WebSocket subscriptions and serverless edge functions in Deno runtime. All edge functions are written using TypeScript and deployed to the global edge infrastructure offered by Supabase. The Groq API is used together with LLaMA 3.3 70B language model as the backbone of AI chat assistant functionality and expense predictions. Resend API serves for sending transactional emails. Charts used in the analytics dashboard come from the Recharts library. Application deployment is done in Vercel and involves deploying continuously from the main Git branch.

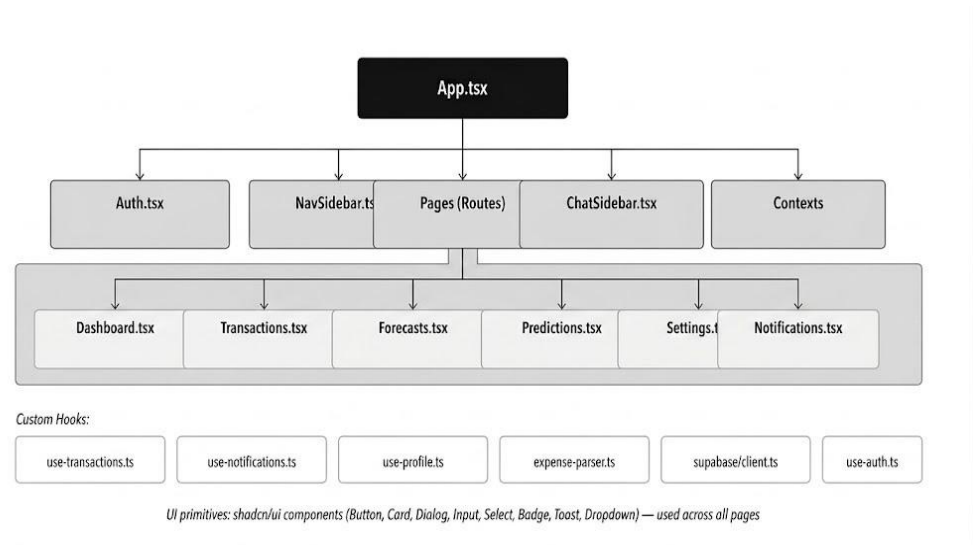


Figure 4.1 React Component Hierarchy

Table 4.1 Technologies

Technology	Category	Purpose
React 18.3 + TypeScript	Frontend	Component-based UI with type safety

Vite 5.4	Build Tool	Fast bundling and hot module replacement
Tailwind CSS	Styling	Utility-first responsive design
Supabase (PostgreSQL)	Backend / DB	Database, auth, real-time, edge functions
Groq API (LLaMA 3.3 70B)	AI	Natural language chat and predictions
Resend API	Email	Transactional budget alert emails
TanStack Query	State	Server state caching and synchronization
Recharts	Visualization	Pie charts and stacked bar charts
Vercel	Deployment	Cloud hosting with CDN

4.2 Database Schema

The database includes three tables defined in Table 4.2. Transactions table stores information about all transactions made and its columns include: id (primary key UUID field, auto-generated), user_id (foreign key, references auth.users table), description (text, non-null), amount (numeric, non-null), category (text, default='Other'), date (date, default=current_date), flagged (boolean, default=false) and created_at (timestampz). The profiles table keeps all user preferences and has the following columns: id, user_id (unique foreign key), display_name, avatar_url, monthly_budget (numeric, default=2000), currency (text, default='USD'), email_daily_reminders (boolean, default=true), email_overspending_alerts (boolean, default=true), created_at, and updated_at. The notifications table has rows about sent budget alerts and includes: id (UUID), user_id, message (text), level (one of text constraints: 'info', 'warning', 'critical'), dismissed (boolean, default=false), and created_at. All monetary values use a numeric data type to avoid problems with floating point precision; timestampz is used in all timestamps.



Figure 4.2 Database Entity Relationship Diagram

Table 4.2 Database Tables – Columns and Data Types

Table	Key Columns	Purpose
transactions	id, user_id, amount, category, date, flagged	Stores all user financial transactions
profiles	user_id, monthly_budget, currency, email flags	User preferences and budget settings
notifications	user_id, message, level, dismissed	In-app budget alert notifications

4.3 API Architecture and Implementation

Two methods are used to implement the API layer. Database-related logic was automatically generated in the PostgREST REST API provided by Supabase. This API automatically generates typed endpoints based on the database schema. All endpoints use JWT authentication and Row

Level Security policies. Frontend communicates with those endpoints using the Supabase JavaScript client. Some complex operations requiring additional API calls to external endpoints are done using Supabase Edge Functions. The five edge functions that are used include chat-analytics, which takes in the user’s message and conversation, fetches the transaction data, and makes a call to the Groq API; predict-spending that creates forecasts using statistics and then invokes Groq for a single sentence insight; send-email, which builds HTML emails and sends them via Resend; send-overspending-email that handles the budget alert emails by validating the user’s preferences and calling send-email;

and daily-reminder-check that loops through all the users that have their daily reminders switched on and sends them an email if there has not been a transaction for the last 24 hours.

4.4 Module-wise Implementation

Authentication Module

All the processes related to user authentication are implemented inside the Supabase Auth service. The Authentication.tsx page calls either `supabase.auth.signInWithPassword` for logins or `supabase.auth.signUp` for registration. In the process of registration, `display_name` is specified as part of the user metadata, which is used by the `handle_new_user` trigger to add an entry in the profiles table. The AuthContext.tsx context listens to authentication events using the `supabase.auth.onAuthStateChange` callback and updates the global user information state whenever there is any change in the user status, such as logging in or out and refreshing the token. All protected routes in App.tsx require a login route redirection.

Transaction Management Module

The Transactions page provides a fully sortable and searchable transaction table. Sorting is handled on the client side using the `Array.sort` function and passing an appropriate comparator for sorting by date, amount, category, and description. The order is two-directional and is flipped when clicking the table header cells. Searching is based on all three mentioned criteria. Inline editing allows editing of amount and category fields right inside the table row and is supported with React Query mutations that update the UI state optimistically.

AI Chat Module

The ChatSidebar component stores a chat message history in a `useState` hook. On sending a new message, the last ten entries together with the latest message are being sent to the chat-analytics edge function to form a system prompt containing the user's full transaction data (up to 500 rows), transaction categories' statistics, totals over the past 30 days, and anomalies (transactions above twice the category average amount). The response message may consist of special blocks such as `$$EXPENSE:{...}$$` for expense recognition, `$$CHART:{...}$$` for chart generation and `$$TABLE:{...}$$` for tables. The frontend's `parseSpecialBlocks` separates the message into these blocks and processes them accordingly. Figure 4.3 describes this process.

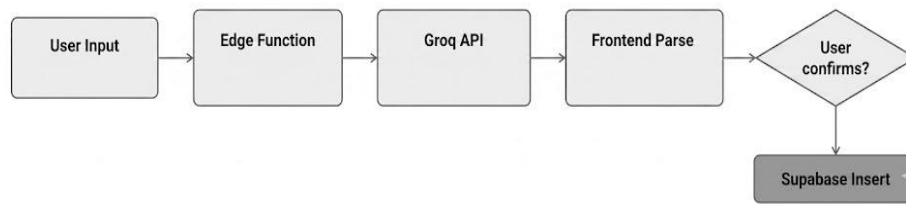


Figure 4.3 AI Chat Expense Logging Flow

Budget Alert Module

The `check_budget_alert` PostgreSQL trigger runs following each insertion or update on the transactions table. The total amount of transactions is calculated during a certain month and is divided by the monthly budget of the user to see if the percentage hits the 80% level (which is considered a warning) or 100% level (critical alert). If the limit has been exceeded, then a new record is created within the notifications table, and a POST request is sent to `send-overspending-email` edge function through `pg_net`. The latter checks if the `email_overspending_alerts` flag of the user is enabled, retrieves the email address by means of the `admin auth API` and sends an email by calling `send_email`. `Send_email` builds an HTML email with a special color palette for each level (warnings have orange headers, whereas critical alerts are marked by deep red) and uses `Resend API` to send emails. Figure 4.4 shows the whole workflow.

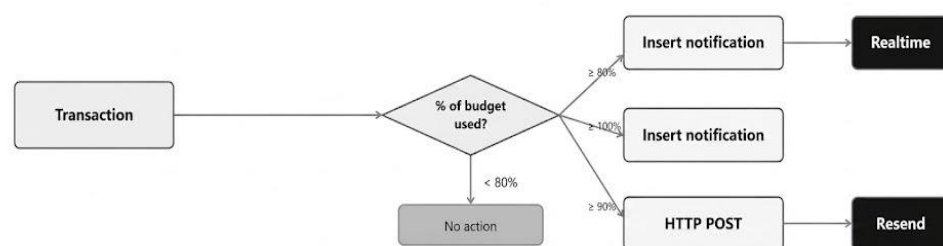


Figure 4.4 Budget Alert Trigger Pipeline

Analytics Module

All analytics calculations occur client-side through the `Forecasts` page, where we make use of `useMemo` hooks that are computed only when underlying transactions and filters change. Pie chart category totals are calculated using the `reducer` to calculate the sum for each category based on filtered transactions. Daily totals for stacked bar chart creation are performed by `groupBy` operations on date and then on category inside each date to create objects of the form `{date, Food: 50, Transport: 20, ...}`. Stacked Bar

components for each category are created dynamically from the set of unique filtered categories and grouped using a single `stackId`. A `CATEGORY_COLOR_MAP` is used in pie chart, bar chart, trend line and merchant list to ensure consistent color mapping.

4.5 Report Generation Workflow

FinBuddy AI generates two types of automated reports: in-app visual reports and email reports. The former reports are created dynamically using client-side aggregation of transactions and rendering using the `Recharts` component library. No server-side computing needs to be performed for such reports since calculations can take place in the client, thanks to the usage of `React Query` cache. Email reports are rendered by the `send-email` edge function as HTML strings using the template literal syntax. There are two templates currently supported, namely the overspending alert template, which is activated at 80% and 100% budget thresholds and the daily reminder template, triggered if there are no transactions for the last 24 hours. Both templates use inline CSS styles supported by major email clients. They contain the user's name, some relevant financial figures and a CTA link to the app.

4.6 Security and Data Integrity

There are several layers involved in securing data in FinBuddy AI. Database layer uses Row Level Security (RLS) policies on all three tables and restricts authenticated users from performing any actions other than `SELECT`, `INSERT`, `UPDATE`, and `DELETE` only on their own rows, with a check on the condition `auth.uid() = user_id`. These policies enforce restrictions at the PostgreSQL engine level and thus cannot be bypassed by client-side or API calls with an anonymous key. Supabase Auth provides signed JWT tokens with an expiration of 1 hour. The `autoRefreshToken` option in the client guarantees token renewal before expiration and prevents silent logouts. Tokens are stored in `localStorage` and passed as Bearer tokens in API requests.

Edge function layer authentication is done using `supabase.auth.getUser(token)` on every incoming JWT token before any process can be made. If some functions require privileged access `SUPABASE_SERVICE_ROLE_KEY` is used. It is kept as a Supabase secret and does not become known at any point by the browser. Both API keys from Groq and Resend are kept as secrets.

To avoid any errors in calculations and have accurate data, `NUMERIC` is used instead of `FLOAT` when dealing with numbers representing monetary value. This way, we can have exact decimals when doing calculations and prevent the loss of significant digits.

Client-side validation of the Amount is done using `parseFloat`, and then it is validated on the server side.

5. RESULTS AND DISCUSSIONS

5.1 Results of Each Module

Authentication Module

The authentication module handles the process of user registration with email verification, login using JWT token generation, and persistent sessions. For new users, the ProtectedRoute component will direct them to the login page as they do not yet have authentication credentials. After successful login, AuthContext will be updated right away through the onAuthStateChange listener, and the user will be directed to the dashboard. In Figure 5.1, one can see the dashboard view post authentication.

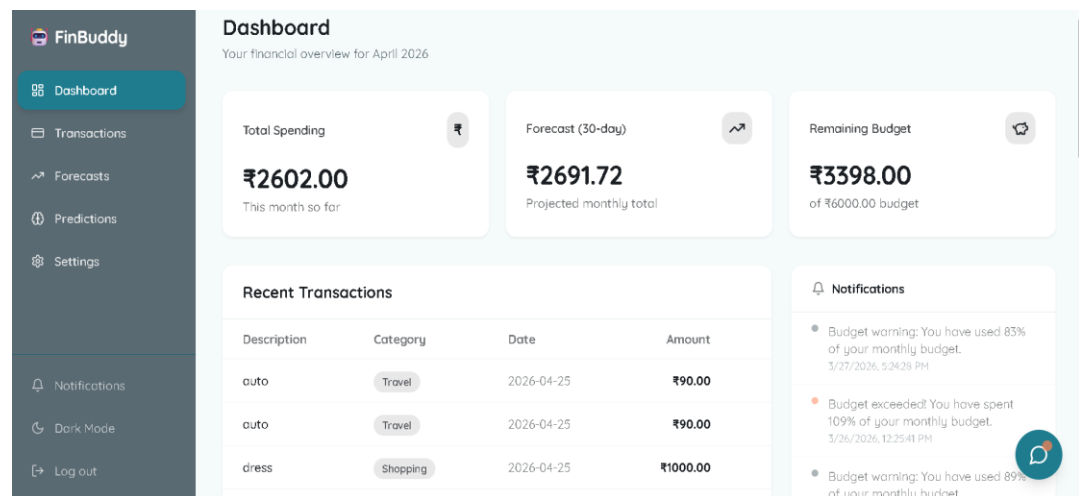


Figure 5.1 Dashboard

Transaction Management Module

All current month transactions will show up in the table sorted according to the date in descending order. The search filter function performs matching across description, category, and date fields. The inline edit feature allows the edits made to the amount and category columns to save changes into the database and update the corresponding row without reloading the page. The flagged column marks the row with an icon whenever a row is determined to be an anomaly based on the AI.

Forecasts and Analytics Module

The Forecasts view generates two kinds of charts: a donut pie chart showing categories of spending as a percentage, and a stacked bar chart showing the category breakdown per day. The consistent use of colors across both graphs will be enabled via the CATEGORY_COLOR_MAP dictionary. Both graphs get updated at the same time through the date range and category selectors. The week-to-week comparison graph shows

totals based on calendar week boundaries. Figure 5.2 shows the outputs generated for the forecasts view.

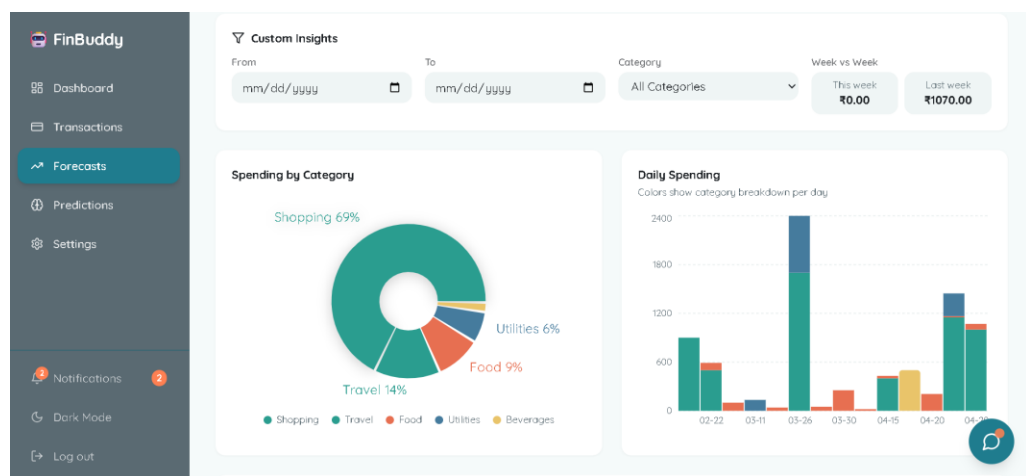


Figure 5.2 Forecasts Page

AI Chat Module

The AI chat assistant processes natural language queries regarding spending, calculates the totals from the transaction data specified within the prompt, and generates a structured expense object to return to the frontend for display in a confirmation message that includes the amount, category, and description of the transaction. The user's response of 'yes' saves the transaction and updates the transaction table instantly, and updates the dashboard. The AI can process voice queries and record expenses in English into the chat input field. Figure 5.3 shows a sample chat exchange.

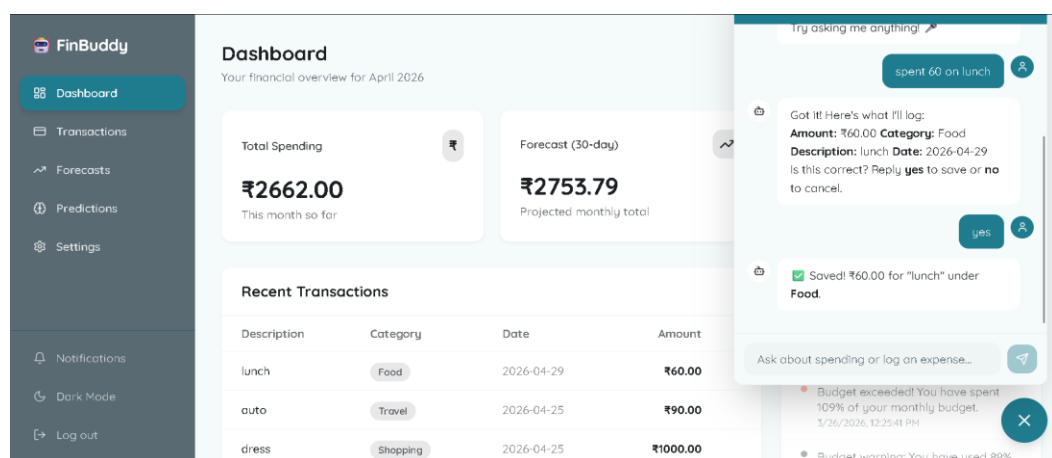


Figure 5.3 AI Chat Assistant Interaction

Budget Alert Module

If the transaction leads to the month's spending exceeding 80% of the budget for that month, then a warning message is added to the database, the Alert Banner is displayed in the dashboard, and a warning email is sent in 8 to 15 seconds. If the

percentage hits 100%, then a critical message and red banner will be displayed, as well as a critical alert email. The sample overspending alert email is shown in Figure 5.4.

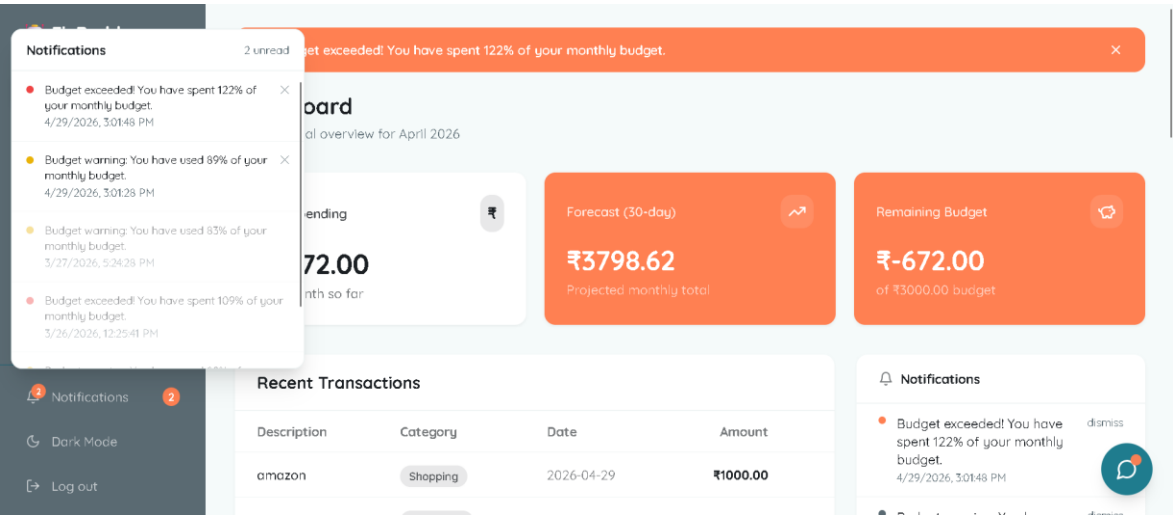


Figure 5.4 Overspending Alert Sample

Predictions Module

The output of the Predictions page accurately presents the estimates for the week (based on daily average multiplied by 7), month (based on daily average multiplied by 30), normal three-month average spending, confidence factor based on months of data, and an artificial intelligence one-line insight generated by Groq. The predictions page output is illustrated in Figure 5.5.

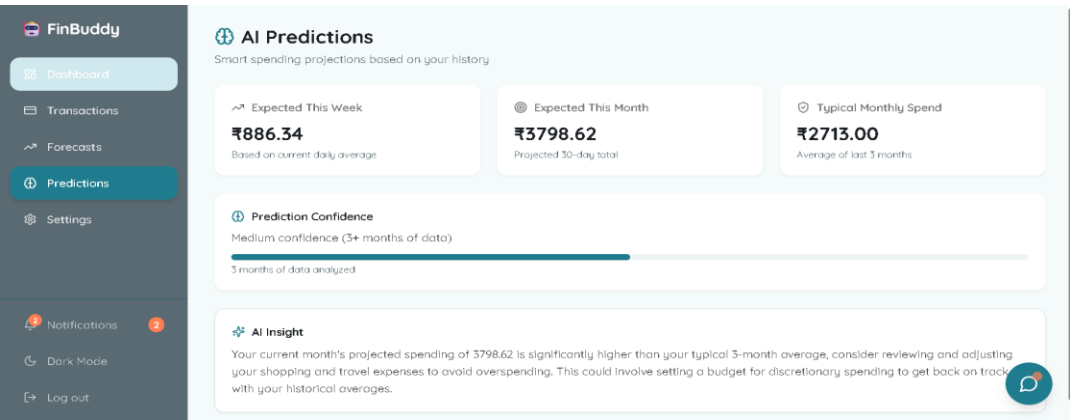


Figure 5.5 Predictions Page

5.2 Performance Measures

Performance assessment was conducted based on the following five dimensions: response time, accuracy, data consistency, reliability of the system, and user accessibility. The performance was tested using transaction datasets of 1000 or fewer with broadband internet access.

Table 5.1 Performance Analysis of System Modules

Metric	Measured Value	Assessment
Authentication latency	~300 ms	Excellent
Transaction query latency	150–400 ms	Good
AI chat response time	1.2–2.5 seconds	Acceptable
Budget alert email delivery	8–15 seconds	Good
Chart rendering (1000 rows)	~60 ms	Excellent
Voice recognition accuracy	>90% (English)	Good
AI expense categorization accuracy	~87%	Good

5.3 Testing and Validation

Three testing phases were performed, namely unit testing of single functions, integration testing for interaction of modules and workflow testing of end-to-end process flows.

Unit Testing

Unit tests were performed through Vitest, which is automatically set up by the application. The pure utility functions of use-transactions.ts file, which include getSpendingTotal, getForecast, getSavings, getCategoryTotal, filterCurrentMonth and getCurrentMonth were tested separately. Also, the natural language inputs of expense parsing functions parseExpense and parseQuery were tested. All unit tests were successful for both normal and edge cases.

Table 5.2 Unit Testing Results

Function Tested	Test Case	Result
getSpendingTotal	Empty array returns 0	Pass

getSpendingTotal	Sums amounts correctly	Pass
filterCurrentMonth	Excludes past months	Pass
parseExpense	“spent 50 on coffee” → Food	Pass
parseExpense	No number → returns null	Pass
getForecast	Daily avg × 30 calculation	Pass

Integration Testing

Integration tests ensured the proper functioning of communication between all the modules. Some major integration tests were as follows: login to dashboard rendering test, chat confirmation to dashboard metric update test, transaction insertion to budget alert test and database insertion to notification test. All integration tests yielded results consistent with expected behavior.

Table 5.3 Integration Testing Results

Integration Scenario	Expected Outcome	Result
Login → Dashboard render	Protected route allows access	Pass
Chat expense confirms → DB insert	Transaction saved, cache refreshed	Pass
Transaction insert → notification	Notification appears in real time	Pass
80% budget crossed → email	Warning email delivered	Pass
Delete transaction → UI update	Table and metrics refresh	Pass

6. CONCLUSION AND FUTURE ENHANCEMENTS

6.1 Conclusion

Financial literacy and managing finances personally still continue to be a major problem that affects millions of people throughout the world. In absence of knowledge about one's expenses, savings capacity, and budget management, people cannot make smart financial decisions. FinBuddy AI is a smart application for tracking personal finances through web applications using full stack development and artificial intelligence.

Also, the case with FinBuddy AI suggests that it is feasible to utilize the Supabase framework for a complete backend without the need for setting up another API server alongside. It gives one more example of the effective implementation of Groq API with LLaMA 3.3 70B model into financial applications, as well as initiating the database events to implement some business operations. The implementation of the notification pipeline by means of triggers from PostgreSQL, pg_net HTTP requests, and Resend Email Service is a great illustration of the event-driven notification system independent of the frontend component.

Lastly, all the seven application goals highlighted earlier have been fulfilled. They include: real-time transaction management, engagement with the AI-powered financial advisor, speech-to-text transcription for inputting the transaction, the budget notification system functioning on two interfaces (80% and 100%), generation of stacked graphs that show expenditure per category, Row Level Security of PostgreSQL, and cloud-native deployment using Vercel.

6.2 Future Scope

- **Gmail API Integration:** Gmail API integration would enable the user to gain access to e-mails that contain transaction information, which would then be parsed with the help of AI regarding the amount, merchant, and transaction category.
- **Bank SMS Processing:** Develop a mobile application that would be able to process bank SMS entries and synchronize them with the main application via Supabase API.
- **Saving Goals:** Enable the users to create saving goals with deadlines. The AI would estimate how much money should be saved per week by the user, as well as provide advice on how to spend less in each spending category.

- Recurring Spending Detection: Ability to automatically detect subscription fees, rent payments, utility expenses, etc., based on the amount and description.
- CSV/PDF Download: Option to download transactions by categories to generate annual tax forms.
- Household Budget Management System: Develop an account-based budget management platform where each family member would be able to enter their transactional data and monitor their contribution to the budget.

REFERENCES

- [1] Deno Documentation (2025) Deno – A Modern Runtime for JavaScript and TypeScript. Available at: <https://docs.deno.com> (Accessed: April 2026).
- [2] Groq Documentation (2025) Groq API Reference. Available at: <https://console.groq.com/docs> (Accessed: April 2026).
- [3] Kearns M. and Nevmyvaka Y. (2013) Machine Learning for Market Microstructure and High Frequency Trading. In: Algorithmic and High-Frequency Trading, O'Reilly Media, pp. 1–21.
- [4] Lalita Parameswari D.V., Seetha M. and Ragha Deepika K. (2014) Fuzzy Bayesian Classification for Spatial Data Streams with P-Trees. International Journal of Advanced Research in Computer and Communication Engineering, Vol. 3(6), pp. 7336–7340.
- [5] Lucide Icons (2025) Lucide – Beautiful and Consistent Icons. Available at: <https://lucide.dev> (Accessed: April 2026).
- [6] React Documentation (2024) React – The Library for Web and Native User Interfaces. Available at: <https://react.dev> (Accessed: April 2026).
- [7] React Router Documentation (2025) React Router v6. Available at: <https://reactrouter.com/en/main> (Accessed: April 2026).
- [8] Recharts Documentation (2025) Recharts – A Composable Charting Library. Available at: <https://recharts.org> (Accessed: April 2026).
- [9] Resend Documentation (2025) Resend – Email API for Developers. Available at: <https://resend.com/docs> (Accessed: April 2026).
- [10] Seetha M. (2023) A Framework for Agricultural Food Supply Chain using Blockchain Technology. Proceedings of the International Conference on Intelligent Computing and Communication, Vol. 978-981-99-1587-3, pp. 18–19.
- [11] Seetha M. (2023) Development and Implementation of Efficient Recruitment Portal Using Blockchain. Advances in Intelligent Systems and Computing, Vol. 2194, pp. 5365–5380.
- [12] Seetha M. (2023) Implementing Digitalization in Square Blockchain Technology Using HDS Methodologies. Proceedings of the International Conference on Intelligent Computing and Communication, Vol. 978-981-99-1587-3, pp. 33–34.
- [13] Supabase Documentation (2025) Supabase – The Open-Source Firebase Alternative. Available at: <https://supabase.com/docs> (Accessed: April 2026).
- [14] Tailwind CSS Documentation (2025) Tailwind CSS – A Utility-First CSS Framework. Available at: <https://tailwindcss.com/docs> (Accessed: April 2026).

[15] TanStack Query Documentation (2025) TanStack Query v5. Available at: <https://tanstack.com/query/v5> (Accessed: April 2026).

[16] TypeScript Documentation (2025) TypeScript – JavaScript with Syntax for Types. Available at: <https://www.typescriptlang.org/docs> (Accessed: April 2026).

[17] Vite Documentation (2025) Vite – Next Generation Frontend Tooling. Available at: <https://vitejs.dev> (Accessed: April 2026).

GLOSSARY

API (Application Programming Interface):

A set of rules and protocols that allows different software applications to communicate and exchange data with one another.

Authentication (Auth):

The process of verifying a user's identity, typically via email and password, before granting access to protected resources.

BaaS (Backend-as-a-Service):

A cloud service model that provides developers with pre-built backend infrastructure including databases, authentication, and serverless functions.

Cache:

A temporary storage layer that holds recently fetched data so it can be returned quickly without a new network request.

CDN (Content Delivery Network):

A distributed network of servers that delivers web content to users from the server geographically closest to them, reducing latency.

CSS (Cascading Style Sheets):

A stylesheet language used to describe the visual presentation of a document written in HTML.

CRUD (Create, Read, Update, Delete):

The four basic operations performed on database records in any data-driven application.

Edge Function:

A serverless function that runs on a distributed network close to the user, reducing latency. In FinBuddy AI, edge functions run on the Deno runtime via Supabase.

JWT (JSON Web Token):

A signed token that encodes a user's identity and expiry claims. Used in FinBuddy AI to authenticate API requests to Supabase and edge functions.

LLM (Large Language Model):

A machine learning model trained on vast amounts of text data, capable of understanding and generating natural language. FinBuddy AI uses LLaMA 3.3 70B via Groq.

Memorization:

An optimization technique that caches the result of an expensive computation and returns the cached value when the same inputs occur again. Used via useMemo in React.

PostgreSQL:

A powerful, open-source relational database management system. Used as Supabase's underlying database engine in FinBuddy AI.

RLS (Row Level Security):

A PostgreSQL feature that enforces access control rules at the database row level, preventing users from reading or writing data belonging to other users.

REST (Representational State Transfer):

An architectural style for designing networked applications using HTTP methods (GET, POST, PATCH, DELETE) to perform operations on resources.

SaaS (Software-as-a-Service):

A software delivery model in which applications are hosted in the cloud and accessed by users over the internet.

TypeScript:

A strongly typed programming language that builds on JavaScript, adding static type definitions to catch errors at compile time rather than at runtime.

UUID (Universally Unique Identifier):

A 128-bit random identifier used as a primary key for database rows. Guaranteed to be globally unique without coordination between systems.

Vite:

A fast frontend build tool and development server that uses native ES modules for near-instant hot module replacement during development.

WebSocket:

A protocol that enables persistent, bidirectional communication between a browser and a server, used in FinBuddy AI for real-time notification subscriptions.

APPENDIX

A. Application File Structure

The FinBuddy AI repository is organized as follows:

- ❖ `src/main.tsx` — Application entry point. Mounts the React app into the Document Object Model (DOM).
- ❖ `src/App.tsx` — Root component. Defines all routes and wraps the app in context providers.
- ❖ `src/index.css` — Global styles, CSS custom properties (color variables), and animation keyframes.
- ❖ `src/components/` — Reusable UI components (NavSidebar, ChatSidebar, MetricCard, AlertBanner, etc.).
- ❖ `src/pages/` — Page-level components (Index, Transactions, Forecasts, Predictions, Settings, Auth).
- ❖ `src/contexts/` — React context providers (AuthContext, ThemeContext, CurrencyContext).
- ❖ `src/hooks/` — Custom React hooks (use-transactions, use-notifications, use-speech-recognition, use-mobile).
- ❖ `src/lib/` — Utility functions (expense-parser, utils).
- ❖ `src/integrations/supabase/` — Supabase client and auto-generated TypeScript types.
- ❖ `supabase/functions/` — Edge functions (chat-analytics, predict-spending, send-email, send-overspending-email, daily-reminder-check).
- ❖ `supabase/migrations/` — SQL migration files for database schema setup and trigger definitions.

B. Environment Variables

The application requires the following environment variables to be set in a `.env` file at the application root:

- ❖ `VITE_SUPABASE_URL` — The URL of the Supabase application (e.g., `https://xxx.supabase.co`).
- ❖ `VITE_SUPABASE_PUBLISHABLE_KEY` — The public anon key for the Supabase application.

The following secrets must be configured in Supabase using the CLI command `supabase secrets set`:

- ❖ `GROQ_API_KEY` — API key from `console.groq.com` for accessing the LLaMA model.
- ❖ `RESEND_API_KEY` — API key from `resend.com` for sending transactional emails.
- ❖ `RESEND_FROM_EMAIL` — Verified sender email address (e.g., `alerts@yourdomain.com`).

C. Deployment Steps

To deploy FinBuddy AI from source, follow these steps:

- ❖ Clone the repository and run `npm install` to install all Node.js dependencies.
- ❖ Create a Supabase application and run the SQL migration files in `supabase/migrations/` via the Supabase dashboard SQL Editor in filename order.
- ❖ Create a `.env` file at the application root and set `VITE_SUPABASE_URL` and `VITE_SUPABASE_PUBLISHABLE_KEY` with values from the Supabase application settings.
- ❖ Set Supabase secrets for `GROQ_API_KEY`, `RESEND_API_KEY`, and `RESEND_FROM_EMAIL` using the Supabase CLI.
- ❖ Deploy all edge functions using the command: `supabase functions deploy`.
- ❖ Connect the repository to Vercel and add `VITE_SUPABASE_URL` and `VITE_SUPABASE_PUBLISHABLE_KEY` as environment variables in the Vercel application settings.
- ❖ Trigger a production deployment by pushing to the main Git branch.